Team Greenlight

DSN x BCT LLM Agent Hackathon 3.0

Technical Solution Paper

User Modelling & Personalised Recommendation System
with Nigerian English Contextualisation

May 2026 | Task A (port 8001) + Task B (port 8002) | FastAPI · FAISS · Gemini 2.5 Flash

1. Executive Summary

Team Greenlight presents a dual-task LLM agent system for the DSN x BCT Hackathon. **Task A** implements personalised review generation using a two-step agentic pipeline: user profile extraction followed by Gemini 2.5 Flash review synthesis with a local sentiment-based rating stage. **Task B** implements a three-step retrieval-augmented recommendation pipeline: LLM-driven intent reasoning, FAISS neural retrieval using all-MiniLM-L6-v2 (384-dimensional embeddings), and LLM reranking with Nigerian Pidgin explanations. Both tasks include authentic Pidgin contextualisation throughout all outputs. Both services are containerised via Docker and exposed as production-grade FastAPI REST APIs.

Component	Technology	Port
Task A — User Modelling	FastAPI + Gemini 2.5 Flash	8001
Task B — Recommendation	FastAPI + FAISS + Gemini 2.5 Flash	8002
Embeddings	all-MiniLM-L6-v2 (sentence-transformers)	—
Vector Index	FAISS IndexFlatIP (cosine-normalised)	—
Container Runtime	Docker (python:3.10-slim base)	—

2. Data Pipeline

2.1 Dataset Sources

Dataset	HuggingFace Source	Records	Domain
Yelp Reviews	Yelp/yelp_review_full	5,000	Restaurants / Businesses
Amazon Grocery	McAuley-Lab/Amazon-Reviews-2023 (Grocery)	3,000	Food Products
Amazon Books	McAuley-Lab/Amazon-Reviews-2023 (Books)	2,000	Books
TOTAL	—	9,999	1,271 user profiles

Note: Goodreads data is no longer publicly distributed. Amazon Books is used as a high-quality substitute with comparable review depth and user diversity, disclosed per competition rules.

2.2 Unified Review Schema

All records are normalised to a common six-field schema before indexing: **user_id**, **rating** (1–5 stars), **review_text**, **item_name**, **item_category**, and **source_dataset**. This unified representation lets both Task A and Task B operate on a single data file (*data/combined_sample.json*).

2.3 User Profile Construction

For each user with two or more reviews a profile is built at data-pipeline time and stored in *data/user_profiles.json*. Each profile captures six fields:

- **avg_rating** — mean star rating across all reviews
- **sentiment_style** — positive / neutral / negative (derived from avg_rating threshold)
- **typical_length** — short (<100 words) / medium / long (>300 words)
- **common_words** — top-20 TF-IDF-style content words with stopwords removed
- **avg_review_length** — mean character count
- **review_count** — total number of reviews contributed

3. Task A — User Modelling Architecture

3.1 API Endpoint

POST /generate-review accepts five fields: *user_id*, *item_name*, *item_category*, *item_description*, and *price_range*. It returns *review_text*, *star_rating* (1.0–5.0), *confidence* (0–1), *user_style_matched*, and *user_found*. A **GET /health** endpoint is also available for Docker health checks.

3.2 Two-Step Agentic Pipeline

Step 1 — Profile-conditioned review generation: Gemini 2.5 Flash receives the user's full profile (sentiment style, typical length, avg rating, top vocabulary) and generates a plain-text review in Nigerian English. Plain text is used instead of JSON schema output mode, which caused double-nested responses when thinking tokens were enabled. thinking_budget is set to 0 to prevent output-token exhaustion on long reviews. Step 2 — Local sentiment rating: The generated review is scored offline using a weighted positive/negative keyword lexicon, then interpolated with the user's historical avg_rating to produce a final 1.0–5.0 star value. This avoids a second Gemini call, which consistently triggered safety filters (finish_reason=2) on rating prompts.

3.3 Nigerian English Contextualisation

The system prompt embeds authentic Pidgin expressions directly into the generation instructions so outputs feel natural rather than translated. Key phrases include: *"I no go lie"* (honest-opinion opener), *"abeg"* (please / emphasis), *"chai!"* (surprise or delight), *"omo"* (street-level enthusiasm), *"na wa o"* (mild frustration or disbelief), *"e dey sweet"* (it is delicious), *"sharp sharp"* (immediately / promptly), and *"wetin"* / *"dem"* (what / them). Usage examples for each phrase are included in the prompt so the model applies them contextually rather than randomly.

3.4 Key Design Decisions

Several engineering choices in Task A emerged from debugging real failure modes during development:

- **Plain-text review, not JSON schema output:** Gemini's *response_mime_type="application/json"* mode caused double-nested JSON objects when thinking tokens were active. Plain-text generation followed by local parsing is more robust.
- **thinking_budget=0:** Default reasoning tokens consumed the output budget and produced truncated reviews. Disabling them keeps the full 2,048 output tokens available for the review text.
- **Local sentiment heuristic for the rating:** A dedicated rating prompt reliably triggered a safety-filter refusal (finish_reason=2). Offline keyword scoring is deterministic, free, and achieves comparable accuracy.
- **avg_rating anchor:** The sentiment score is interpolated with the user's historical average rating so a habitually harsh reviewer still produces sub-4-star predictions even when their generated text sounds positive.

4. Task B — Recommendation Architecture

4.1 API Endpoints

POST /recommend (warm-start) — accepts *user_id*, *conversation_history*, and *top_k*. Looks up the user profile, runs FAISS retrieval over the user's past review texts, and returns personalised recommendations with Nigerian Pidgin explanations.

POST /recommend/cold-start (new users) — accepts *item_category*, *description_of_what_i_want*, and *top_k*. Skips profile lookup entirely and uses the user-supplied description as the FAISS query directly.

4.2 Three-Step Agentic Pipeline

Step 1 — Intent reasoning: Gemini extracts a concise search query from the user's profile and conversation history. Step 2 — FAISS neural retrieval: all-MiniLM-L6-v2 encodes the query into a 384-dimensional vector; FAISS IndexFlatIP retrieves the top-50 cosine-similar items from 9,999 pre-indexed reviews. For warm-start users the query is built from the user's actual past review texts (not a generic LLM summary) — this grounds retrieval in demonstrated preferences and achieves 90% recall of ground-truth items in the top-50 candidates. Step 3 — LLM reranking: Gemini selects and reorders the top candidates, adding a Nigerian Pidgin explanation for each. If Gemini is unavailable the system degrades gracefully to FAISS ranking order, still achieving 65% Hit Rate@5.

4.3 FAISS Index Design

Property	Value / Detail
Embedding model	all-MiniLM-L6-v2 (sentence-transformers)
Vector dimension	384
Index type	IndexFlatIP — exact inner product (cosine after L2 normalisation)
Index size	9,999 vectors, pre-built and cached to data/faiss_index.pkl
Build time	~20 minutes on CPU (one-off); subsequent starts load in under 2 seconds
Query latency	Under 50 ms per search after cache is loaded
Retrieval k	top-50 candidates retrieved; top-15 passed to the reranker
Fallback	scikit-learn TF-IDF if sentence-transformers are unavailable

4.4 Reranking Output Format

Early iterations used JSON for the reranking response. When Gemini's output budget was exhausted mid-object the entire result became unparseable. The final design uses a line-by-line delimiter format so any fully-written item is parseable even if the response is cut short:

```
RANK: 1 NAME: [exact item name from candidates] CAT: [category] SCORE: [0.0-1.0]
SOURCE: [dataset] WHY: [Nigerian Pidgin explanation] ---
```

A critical instruction in the reranking prompt requires the model to copy item names character-for-character from the candidate list. This prevents the model from paraphrasing item IDs, which was the root cause of zero Hit Rate in early evaluation.

4.5 Key Design Decisions

The most impactful engineering choice in Task B was switching the retrieval query from a Gemini-generated need summary to the user's actual past review texts. Semantic search over what a user has genuinely written outperforms a generic LLM description by a wide margin — the Hit Rate@5 rose from 0% to 65% and recall@50 reached 90%.

Other notable decisions:

- **Candidates expanded to top-50:** Wider FAISS retrieval gives the reranker better raw material without meaningfully increasing latency.
- **SentenceTransformer cached globally:** The encoder is instantiated once at module level to avoid a costly reload on every request.
- **Graceful degradation:** All reranker calls are wrapped in try/except. If Gemini fails (e.g. quota exhausted) the system falls back to raw FAISS order and still serves results.
- **Cold-start via direct query:** New users skip the profile lookup; their free-text description becomes the FAISS query directly, achieving 100% cold-start success in evaluation.

5. Evaluation Results

5.1 Task A — Review Generation (16 users)

The evaluation held out each user's most recent review as ground truth. The agent generated a new review for the same item without access to the ground truth. 16 users completed successfully; 4 additional users were skipped due to Gemini free-tier rate limits (20 requests per day).

Metric	Score	Notes
Avg ROUGE-L	0.1357	Vocabulary overlap with held-out reviews. Scores of 0.10–0.15 are typical for open-ended generative tasks where paraphrasing is expected.
RMSE (star rating)	0.8725	Mean squared error on 1–5 star scale. Largest errors occur when sentiment signal is ambiguous.
MAE (star rating)	0.6625	Mean absolute error. The majority of predictions fall within 1 star of ground truth.
Style match rate	100%	User profile was found and applied for every test user.
Avg review length	1,142 chars	Generated reviews are detailed and substantially longer than many ground truth samples.

Individual Results (16 users)

User ID	ROUGE-L	Pred ★	Act ★	Error
yelp_user_0228	0.1193	3.9	3.0	0.90
yelp_user_0051	0.1206	4.2	4.0	0.20
yelp_user_0563	0.1453	3.8	4.0	0.20
yelp_user_0501	0.1176	4.1	2.0	2.10
yelp_user_0457	0.1086	4.0	3.0	1.00
yelp_user_0285	0.1296	3.0	2.0	1.00
yelp_user_0209	0.1270	5.0	4.0	1.00
amazon_AGBJ...7VK	0.2936	5.0	5.0	0.00
yelp_user_0178	0.0818	3.4	3.0	0.40
yelp_user_0191	0.0754	4.3	3.0	1.30
yelp_user_0447	0.1160	4.1	3.0	1.10
yelp_user_0476	0.1289	4.8	4.0	0.80
amazon_AEVP...I2U	0.1143	4.7	5.0	0.30
gr_AH4O...VTS	0.1871	5.0	5.0	0.00
yelp_user_0054	0.0905	3.7	4.0	0.30

amazon_AH2I...XY	0.2154	5.0	5.0	0.00
AVERAGE	0.1357	—	—	0.6625

User IDs abbreviated for display. Full IDs: amazon_AGBJABSJLN7H7AALE7VK, amazon_AEVPPTMG43C6GWSR7I2U, gr_AH4O5W3EM4CKQGHMBVTS, amazon_AH2IW7MZIXM53IRDZ2XY.

5.2 Task B — Recommendation (20 users + 5 cold-start)

Full evaluation across all 20 test users. The system ran primarily on FAISS fallback due to API rate limits, demonstrating that review-text retrieval alone achieves strong results without LLM reranking.

Metric	Score	Notes
Hit Rate@5	0.6500 (65%)	Ground truth item in top 5 for 13 of 20 users.
NDCG@5	0.2968	Relevant items surface near position 1 in the ranked list.
Cold-start success	5/5 (100%)	All 5 new users received relevant recommendations.
Users evaluated	20/20	Full test set completed without any failures.

The key insight: using the user's actual past review texts as the FAISS query (rather than a Gemini-generated need summary) pushed recall@50 to 90% and Hit@5 from 0% to 65%. This is because real review text directly encodes demonstrated taste, while a generic LLM summary loses specificity.

6. System Architecture & Deployment

6.1 Project Structure

The repository is organised into two independent service directories, a shared data directory, evaluation notebooks, and results storage:

```
greenlight/ task_a/ Task A API — User Modelling (port 8001) agent.py Gemini pipeline + local sentiment rating main.py FastAPI routes + Pydantic models Dockerfile task_b/ Task B API — Recommendation (port 8002) agent.py FAISS retrieval + Gemini reranking main.py FastAPI routes Dockerfile data/ combined_sample.json 9,999 unified reviews user_profiles.json 1,271 user profiles faiss_index.pkl pre-built FAISS index + item metadata notebooks/ build_data.py data pipeline (run once) evaluate_task_a.ipynb ROUGE-L + RMSE evaluation evaluate_task_b.ipynb Hit Rate@5 + NDCG@5 evaluation results/ task_a_results.json saved evaluation output task_b_results.json saved evaluation output docker-compose.yml .env GEMINI_API_KEY (not committed)
```

6.2 Docker & Deployment

Both services use a *python:3.10-slim* base image. Task B's Dockerfile pre-downloads the sentence-transformer model at build time so the container starts fully self-contained with no network dependency at runtime. A single *docker-compose.yml* at the project root orchestrates both services with a shared read-only data volume and passes GEMINI_API_KEY from the host environment.

Additional API design notes:

- **UTF-8 JSON patch:** JSONResponse.render is monkey-patched with ensure_ascii=False so the Naira symbol (₦) renders correctly in all clients.
- **CORS middleware:** Enabled on both services for browser-based testing and frontend integration.
- **FAISS cache:** The index is built once (~20 min) and pickled to *data/faiss_index.pkl*. Subsequent container starts load it in under 2 seconds.
- **TF-IDF fallback:** If sentence-transformers are unavailable, Task B automatically falls back to scikit-learn TF-IDF retrieval without any code changes.
- **Pydantic validation:** All request inputs are validated with field_validators; the API returns helpful 422 errors for bad inputs.
- **Health endpoints:** GET /health on both services; Docker HEALTHCHECK is configured with appropriate start_period delays (30 s for Task A, 60 s for Task B which needs extra time to load the FAISS index).

7. Limitations and Future Work

Current Limitations

- **Rating accuracy:** The local sentiment heuristic occasionally misses context-dependent signals such as sarcasm or cultural idioms. A fine-tuned sentiment classifier would improve RMSE meaningfully.

• **ROUGE-L ceiling:** Generative reviews inherently score low on n-gram overlap because the model paraphrases rather than reproduces text verbatim. Semantic similarity metrics such as BERTScore would better capture generation quality.

• **Gemini free-tier quota:** 20 requests per day limited evaluation throughput. Pay-as-you-go billing resolves this at an estimated cost of under $0.20 for the full evaluation suite.

• **Goodreads unavailability:** The original dataset was removed from public distribution. Amazon Books provides comparable quality but covers a narrower domain.

Future Improvements

• Switch to **gemini-2.5-pro** for higher-quality review generation and more nuanced reranking explanations.

• Add **collaborative filtering** signals alongside semantic similarity for Task B (users who liked X also liked Y).

• Expand Pidgin vocabulary with a curated Nigerian slang lexicon and evaluate outputs with native speakers.

• Implement **streaming responses** (Server-Sent Events) for Task A to reduce perceived latency on long reviews.

• Evaluate with **BERTScore** in addition to ROUGE-L to better capture semantic quality of generated reviews.

8. References

1. Yelp Open Dataset — https://huggingface.co/datasets/Yelp/yelp_review_full

2. McAuley-Lab Amazon Reviews 2023 — https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023

3. Reimers & Gurevych (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.

4. Johnson, Douze & Jégou (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data.

5. Google Gemini API Documentation — https://ai.google.dev/gemini-api/docs

6. FastAPI Documentation — https://fastapi.tiangolo.com